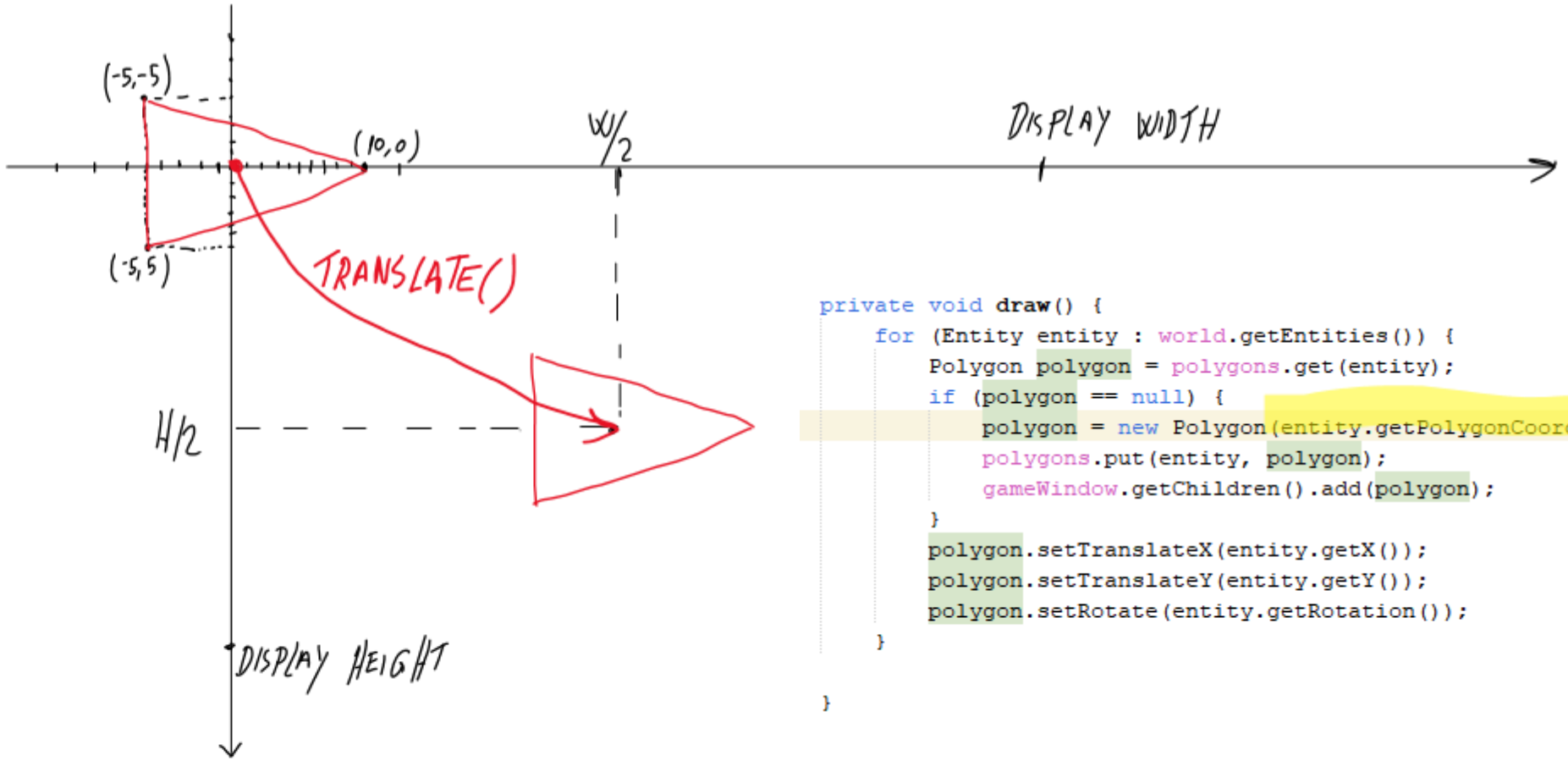
CBSE 2D Games

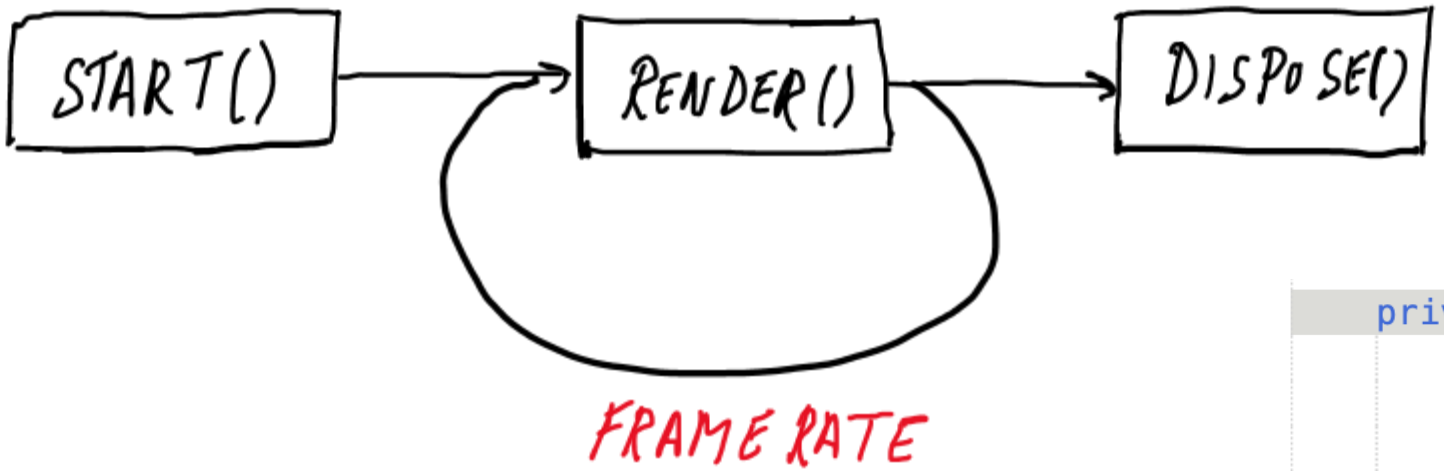
Client Coordinates




```
private void draw() {
    for (Entity entity : world.getEntities()) {
        Polygon polygon = polygons.get(entity);
        if (polygon == null) {
            polygon = new Polygon(entity.getPolygonCoordinates());
            polygons.put(entity, polygon);
            gameWindow.getChildren().add(polygon);
        }
        polygon.setTranslateX(entity.getX());
        polygon.setTranslateY(entity.getY());
        polygon.setRotate(entity.getRotation());
    }

}
```

Game Loop

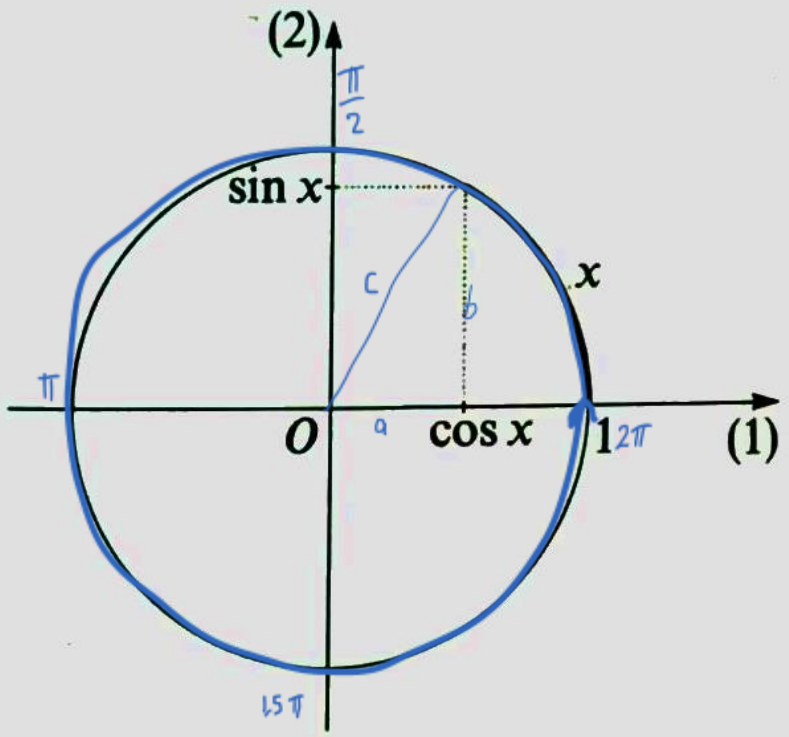



```
    private void render() {
        new AnimationTimer() {
            private long then = 0;

            @Override
            public void handle(long now) {
                update();
                draw();
                gameData.getKeys().update();
            }

        }.start();
    }
```

Trigonometry



Aflæsning af cos x og sin x

PYTHAGORAS

(73) $(\cos x)^2 + (\sin x)^2 = 1$

(74) $\cos(x + 2\pi) = \cos x$
$\sin(x + 2\pi) = \sin x$

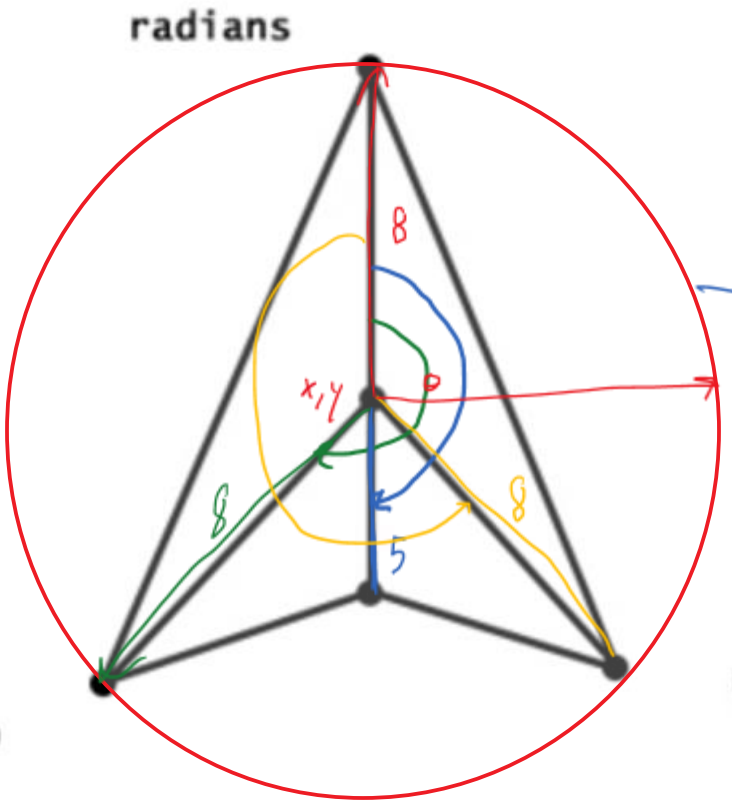
(75) $\cos(-x) = \cos x$
$\sin(-x) = -\sin x$

(76) $\cos(\pi - x) = -\cos x$
$\sin(\pi - x) = \sin x$

(77) $\cos\left(\frac{\pi}{2} - x\right) = \sin x$

$\sin\left(\frac{\pi}{2} - x\right) = \cos x$

Ship




```
shapex[0] = x + MathUtils.cos(radians) * 8;
shapey[0] = y + MathUtils.sin(radians) * 8;

shapex[1] = x + MathUtils.cos(radians - 4 * 3.1415f / 5) * 8;
shapey[1] = y + MathUtils.sin(radians - 4 * 3.1145f / 5) * 8;

shapex[2] = x + MathUtils.cos(radians + 3.1415f) * 5;
shapey[2] = y + MathUtils.sin(radians + 3.1415f) * 5;

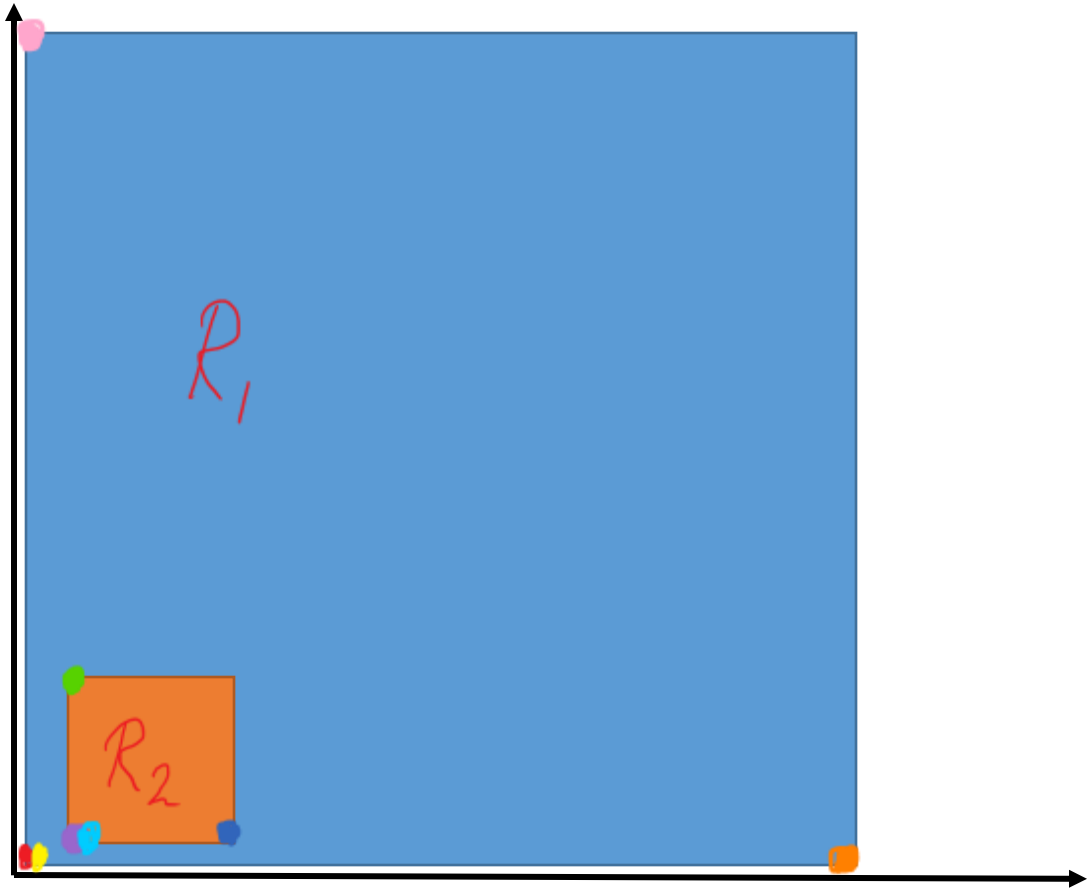
shapex[3] = x + MathUtils.cos(radians + 4 * 3.1415f / 5) * 8;
shapey[3] = y + MathUtils.sin(radians + 4 * 3.1415f / 5) * 8;
```

Box Collision

```
var rect1 = {x: 5, y: 5, width: 50, height: 50}
var rect2 = {x: 20, y: 10, width: 10, height: 10}

if (rect1.x < rect2.x + rect2.width &&
    rect1.x + rect1.width > rect2.x &&
    rect1.y < rect2.y + rect2.height &&
    rect1.height + rect1.y > rect2.y) {
     // collision detected!
}
```

TRUE
TRUE
TRUE
TRUE

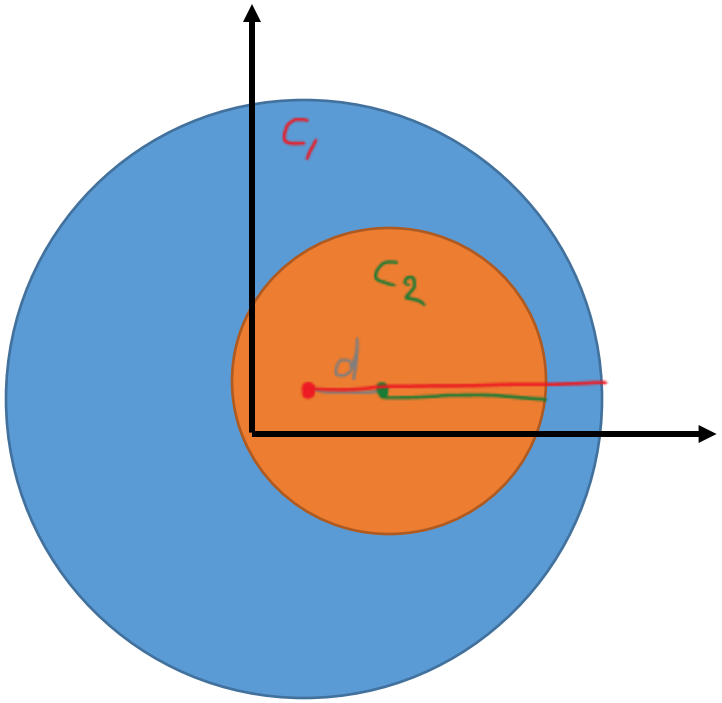


Circle Collision

```
var circle1 = {radius: 20, x: 5, y: 5};
var circle2 = {radius: 12, x: 10, y: 5};

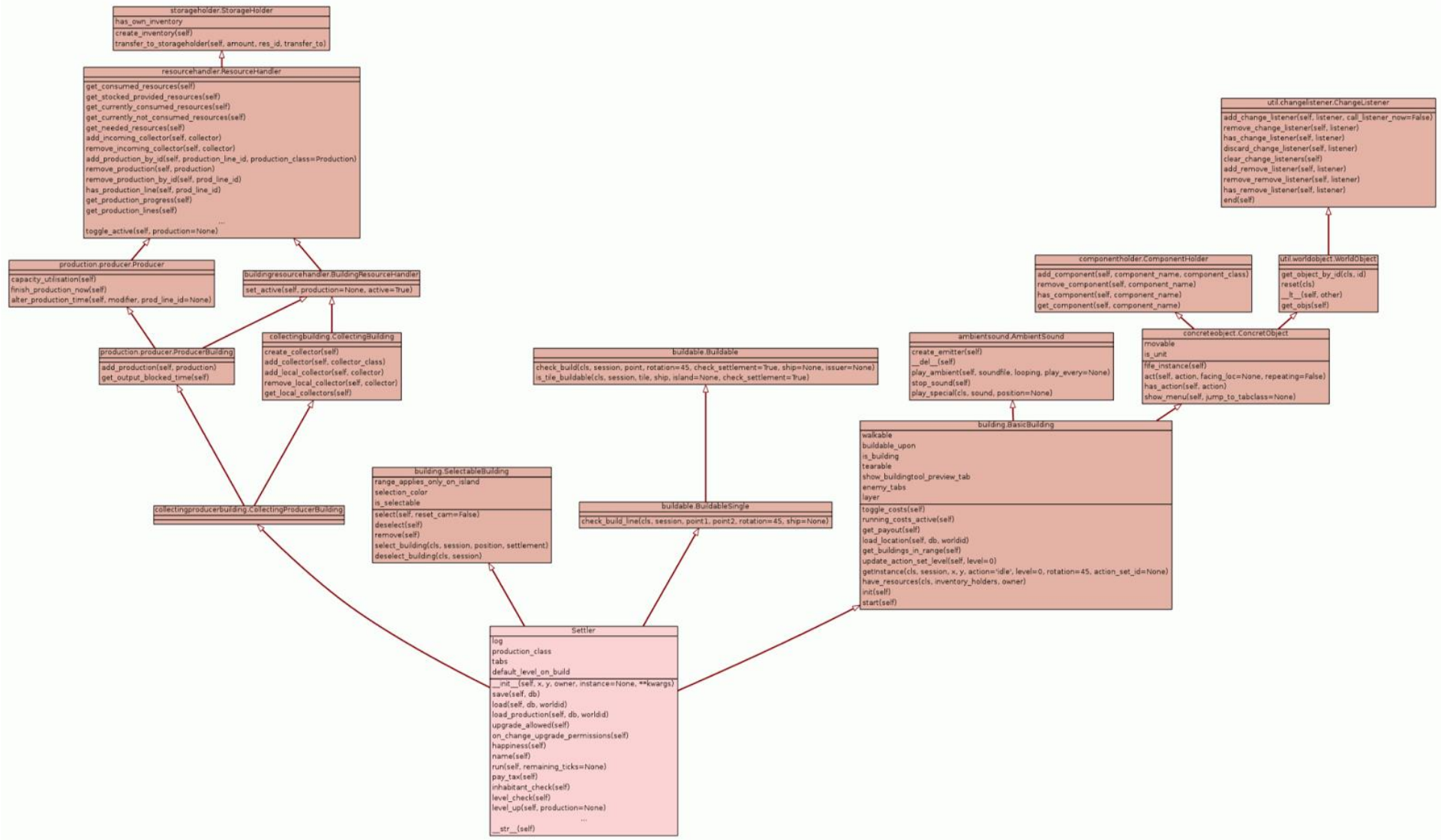
var dx = circle1.x - circle2.x;
var dy = circle1.y - circle2.y;
var distance = Math.sqrt(dx * dx + dy * dy);

if (distance < circle1.radius + circle2.radius) {
    // collision detected!
}
```



Component and DATA-Oriented Game Design

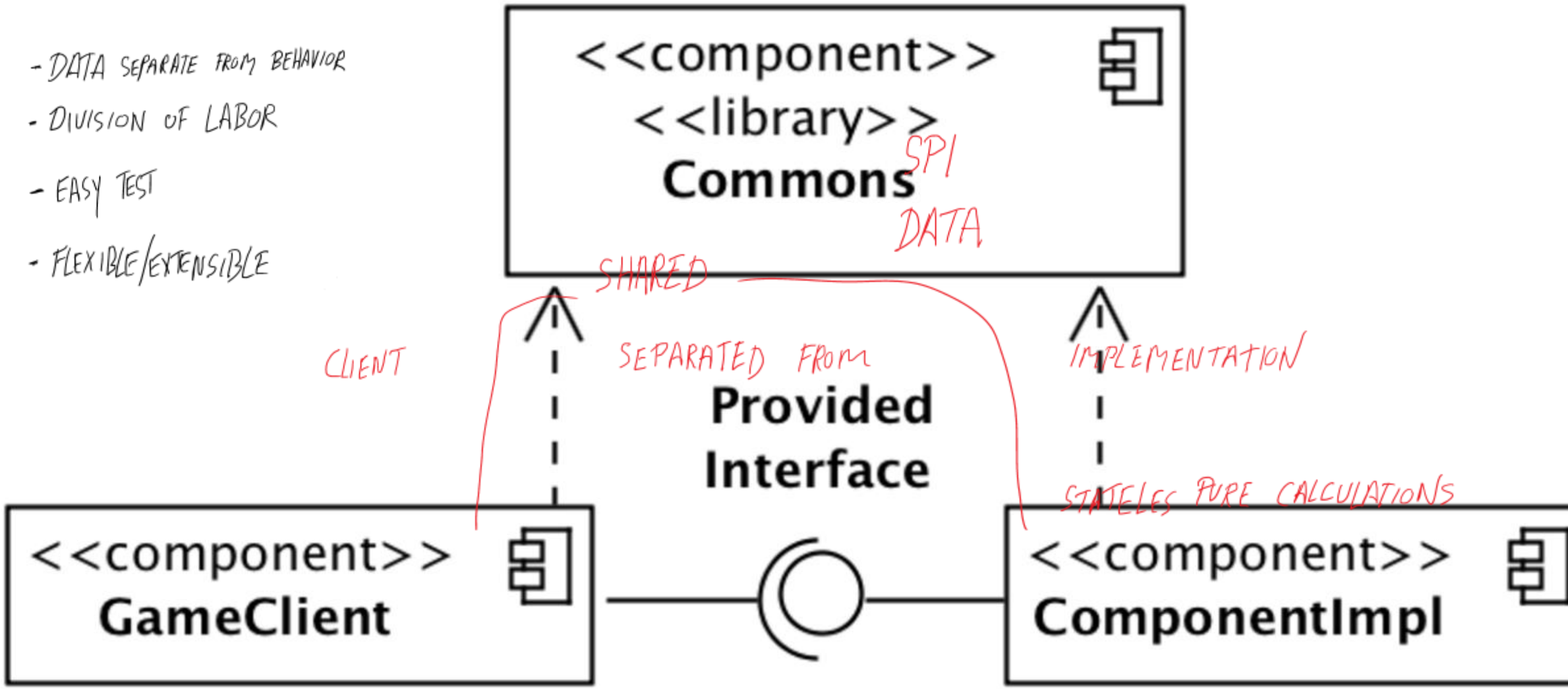
Inheritance based entities



Component-based

FAVOR COMPOSITION OVER INHERITANCE

- DATA SEPARATE FROM BEHAVIOR
- DIVISION OF LABOR
- EASY TEST
- FLEXIBLE/EXTENSIBLE



Entities (POJOS)

```
public final class Entity implements Serializable {

    private final UUID ID = UUID.randomUUID();
    private EntityType type;
    private float x;
    private float y;
    private float dx;
    private float dy;
    private float radians;
    private float maxSpeed;
    private float acceleration;
    private float deacceleration;
    private float[] shapeX;
    private float[] shapeY;
    private int rotationSpeed;
```

UNIQUE ID

PLAYER, ENEMY...

VECTOR DATA

USED FOR DRAWING

Provided Interfaces

(COMMON)

```
public interface IEntityProcessingService {

    void process(GameData gameData, World world);
}
```

DELTA, W, H VECTOR DATA

```
public interface IGamePluginService {
    void start(GameData gameData, World world);
    void stop(GameData gameData, World world);
}
```

ADD DATA ON COMPONENT INSTALL

REMOVE/CLEANUP ON UNINSTALL

Contracts

SPI	**GameEngineSPI**
Operation:	`update(int delta)` SIGNATURE
Description:	Called when at least updateRate ms have elapsed since the last update call. Input-handling, physics, and game logic should be performed in this method. It is also appropriate to update the structure of the scene graph here (adding, removing layers). You should not animate scene graph properties in update; that should happen in paint(float). DO NOT LEAK ANY IMPL. DETAILS!!!
Parameters:	`delta` - time (in ms) since the last update call. This will always be updateRate, unless the game is running slowly. In that case, multiple update calls will be coalesced into a single update call and `delta` will be a multiple of updateRate. Your game should decide at that point whether to run one simulation update with a larger time interval, or to run multiple simulation updates with the standard interval. Beware that if your update calculations take longer than updateRate to complete then your game loop will fall further and further behind and will eventually fail. Choose a long enough update interval to accommodate your processing.
Preconditions:	updateRate ms have elapsed since the last update call INVARIANTS AND CONDITIONS BEFORE CALL
Postconditions:	Structure of the scene graph has been updated. That is, layers has been added or removed. CONDITIONS AFTER CALL